

Estruturas de Dados: Árvores

Uma análise dos diferentes tipos de estruturas de árvores

Felipe Casagrande

Bruno Goulart

Diego Cosme

Renata Mendes

Thiago Moura

Bruno Pires

Lucas Borba

Pedro Lucas

Josias Andrade

Eduardo Rodinei

O que são Estruturas de Dados em Árvore?

Uma **árvore** é uma estrutura de dados não linear, hierárquica, que consiste em nós conectados por arestas. Diferente das estruturas lineares como arrays e listas, as árvores organizam dados de forma hierárquica.

Terminologia básica:

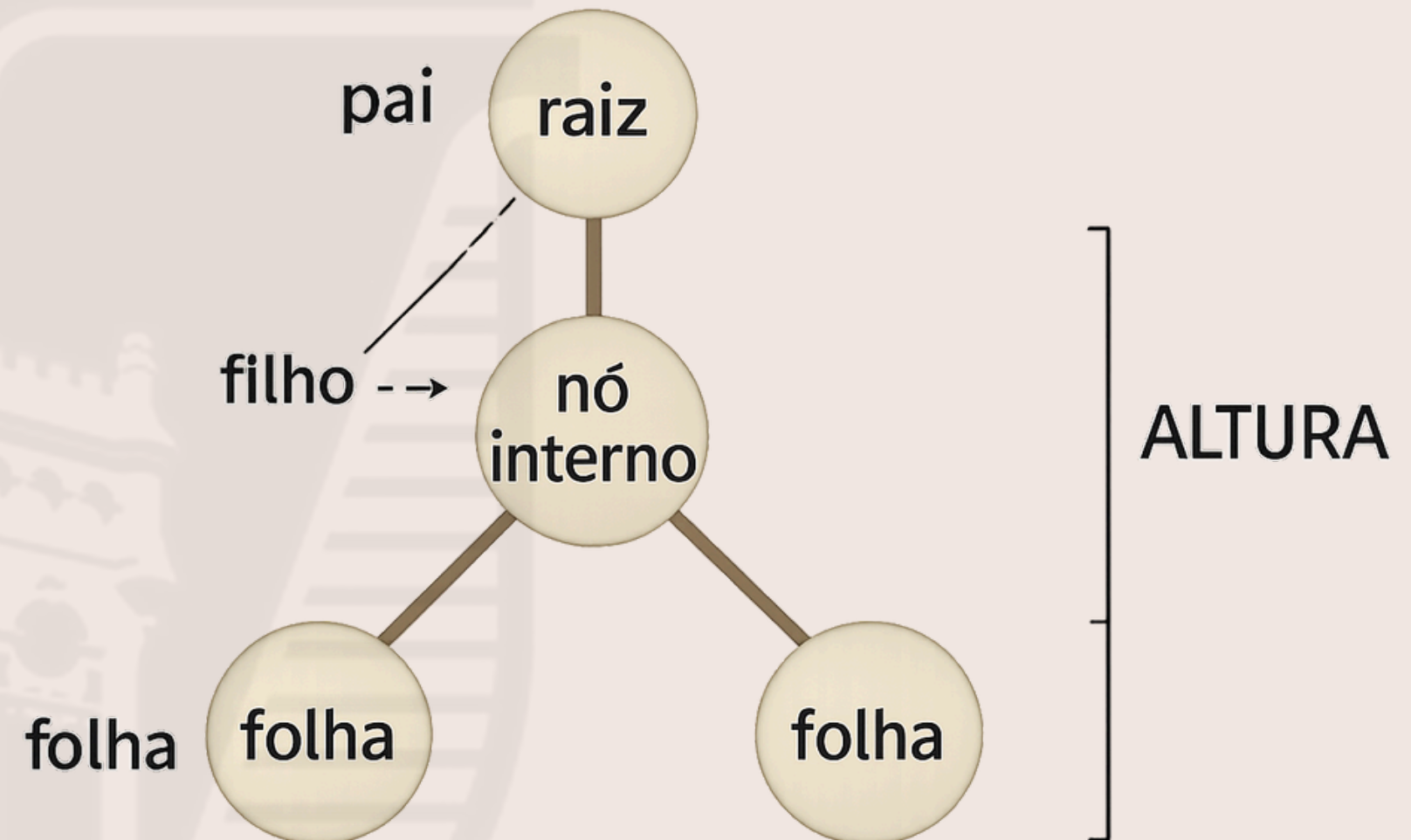
Raiz: O nó superior da árvore, sem pai.

Nó: Elemento que contém dados e referências.

Filho: Nó que possui um pai.

Folha: Nó sem filhos.

Altura: Caminho máximo da raiz até uma folha.



Aplicações

Sistemas de arquivos Compressão de dados Inteligência artificial

Bancos de dados Roteamento em redes Compiladores

Definição

Uma **Árvore Binária** é uma estrutura de dados em que cada nó pode ter **no máximo dois filhos**, geralmente chamados de filho esquerdo e filho direito.

Na **Árvore Binária de Busca (ABB)**, para qualquer nó:

Todos os valores na subárvore esquerda são **menores** que o valor do nó

Todos os valores na subárvore direita são **maiores** que o valor do nó

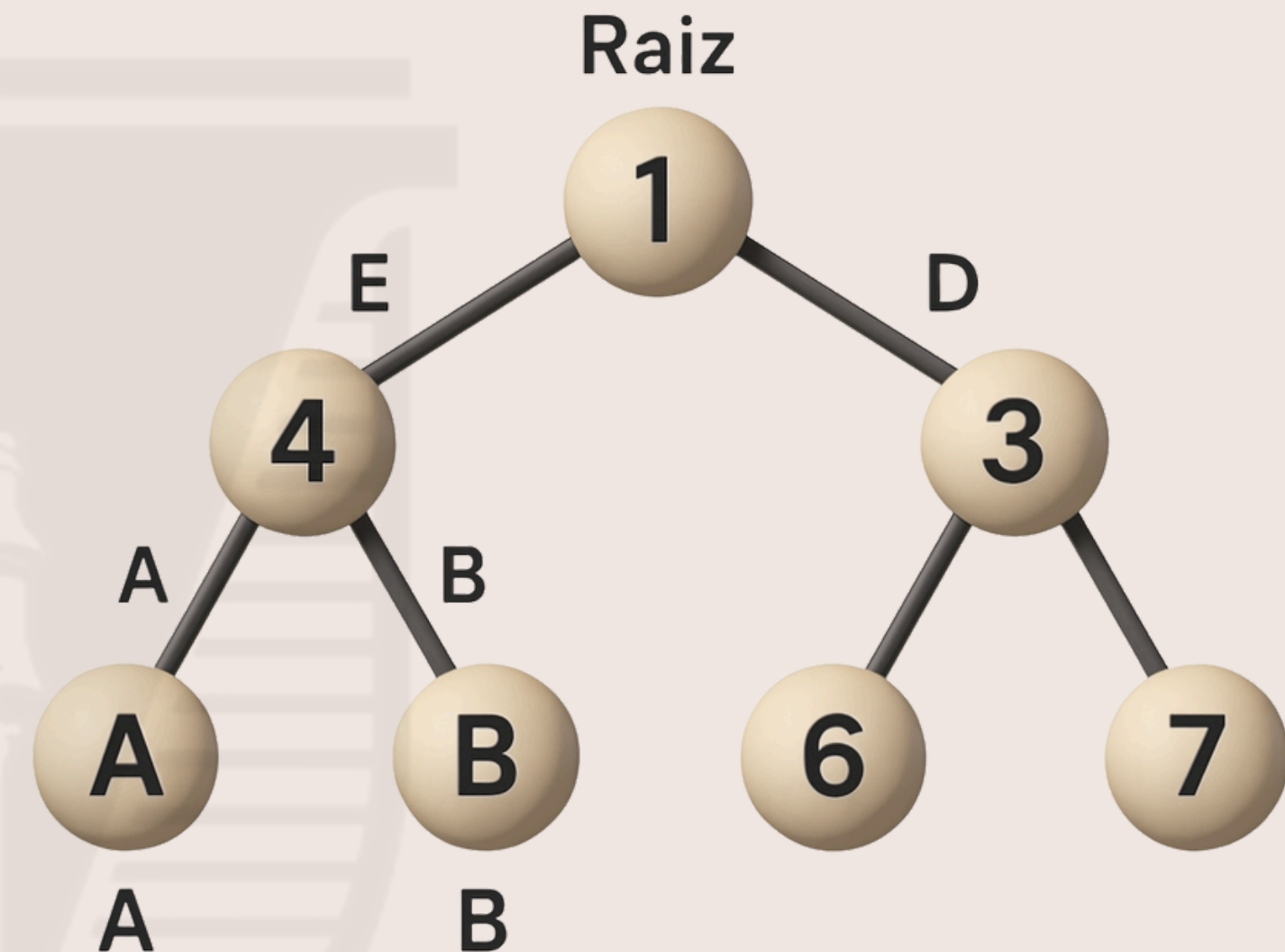
Vantagens e Desvantagens

Vantagens:

Busca eficiente: **$O(\log n)$** em média
Inserção e remoção relativamente simples
Mantém os dados ordenados

Desvantagens:

Pode degenerar em uma lista ligada ($O(n)$)
Não é auto-balanceada



O que são Árvores AVL?

As **Árvores AVL** (Adelson-Velsky e Landis) são árvores binárias de busca **auto-balanceadas**, onde a diferença de altura entre as subárvores esquerda e direita de qualquer nó não pode ser maior que 1.

Características principais:

Mantém um **fator de balanceamento**

para cada nó (diferença entre altura da subárvore esquerda e direita)

O fator de balanceamento deve ser -1, 0 ou 1

Quando o balanceamento é violado, são realizadas **rotações** para restaurar o equilíbrio

Vantagens e Desvantagens

Vantagens:

- Buscas muito rápidas
- Altura sempre balanceada
- Desempenho previsível

Desvantagens:

- Inserções e remoções mais complexas
- Maior overhead de memória
- Rotações podem ser custosas

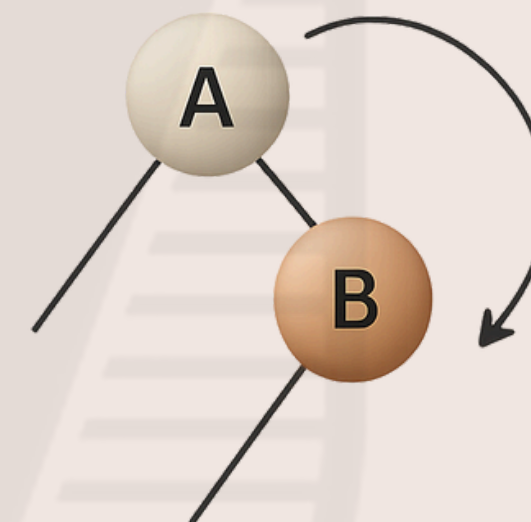
Tipos de rotações:

Rotação Simples:
Esquerda ou direita

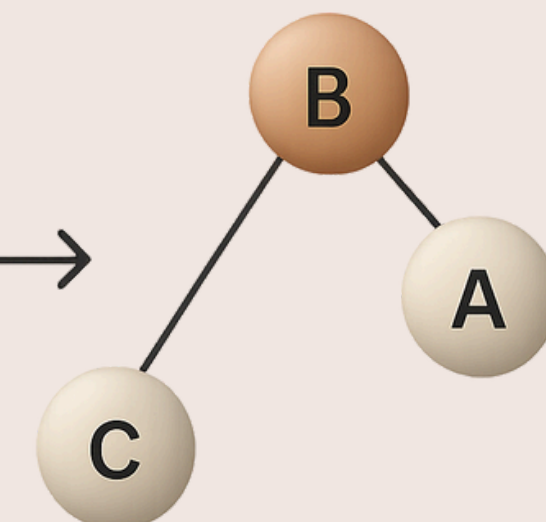
Rotação Dupla:
Combinação de duas rotações

ROTAÇÃO À DIREITA SIMPLES

ANTES



DEPOIS



Exemplos de rotações em Árvores AVL para manter o balanceamento

O que são Árvores Rubro-Negras?

Uma **árvore rubro-negra** é um tipo de árvore binária de busca balanceada onde cada nó possui uma cor (vermelha ou preta) e segue um conjunto de regras que garantem que a árvore permaneça balanceada após inserções e remoções.

Regras das Árvores Rubro-Negras

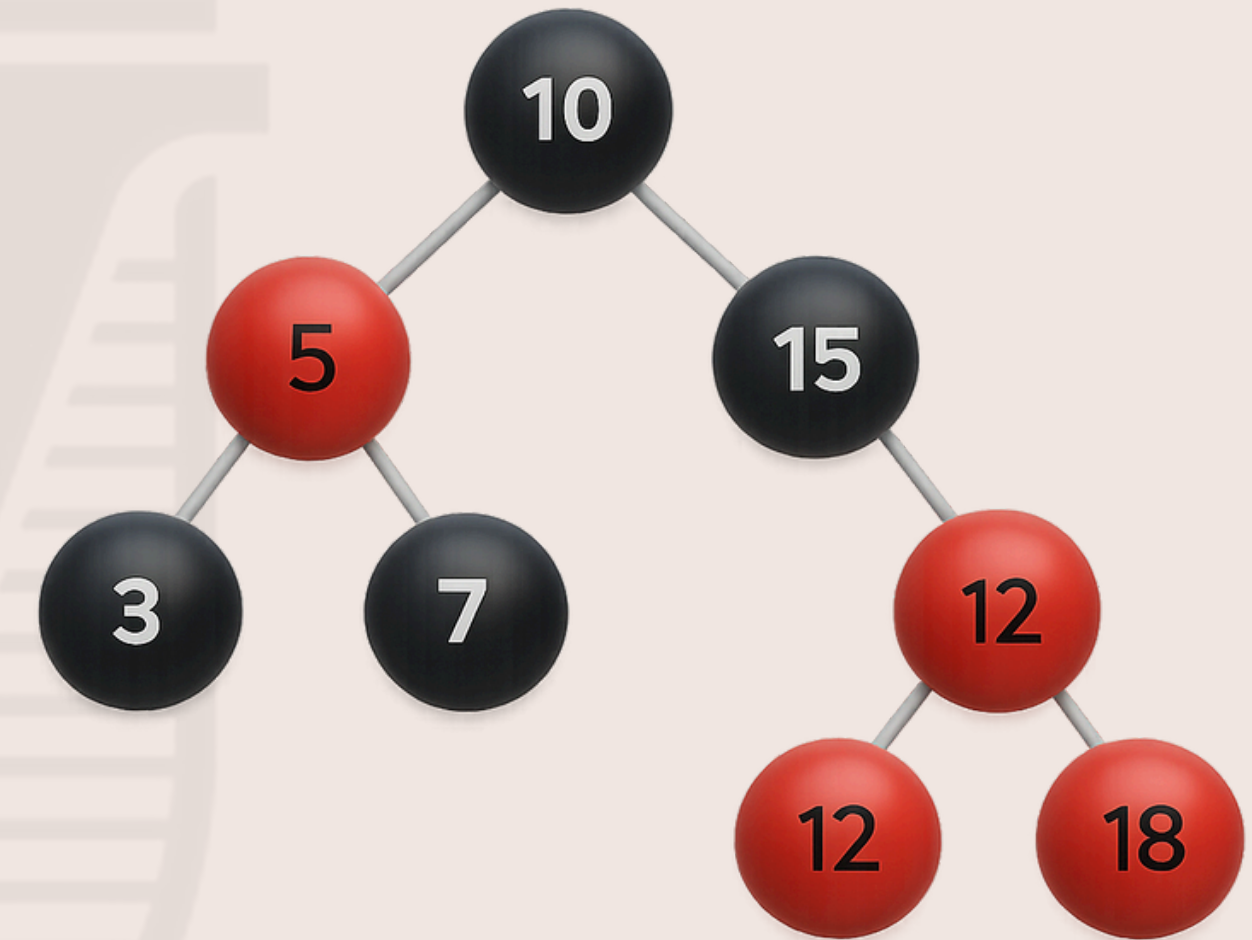
Todo nó é vermelho ou preto.

A raiz é sempre preta.

Todas as folhas (NIL/NULL) são pretas.

Se um nó é vermelho, seus filhos são pretos.

Todo caminho de um nó até suas folhas descendentes contém o mesmo número de nós pretos.



ÁRVORE RUBRO-NEGRA

Vantagens e Desvantagens

Vantagens:

- Operações em $O(\log n)$
- Menos rotações que AVL
- Eficiente para inserções

Desvantagens:

- Implementação complexa
- Maior uso de memória

O que são Árvores Digitais?

Uma **Árvore Digital** (ou **Trie**, do inglês "retrieval") é uma estrutura de dados do tipo árvore ordenada, usada para armazenar um conjunto de strings, onde a posição do nó na árvore define a chave com a qual ele está associado.

Cada nó representa um prefixo comum

Os nós não armazenam a chave completa, apenas parte dela

A raiz representa a string vazia

Vantagens e Aplicações

Vantagens:

Busca por prefixos eficiente - $O(m)$

Economia de espaço

Verificação rápida de prefixos

Estrutura flexível

Aplicações:

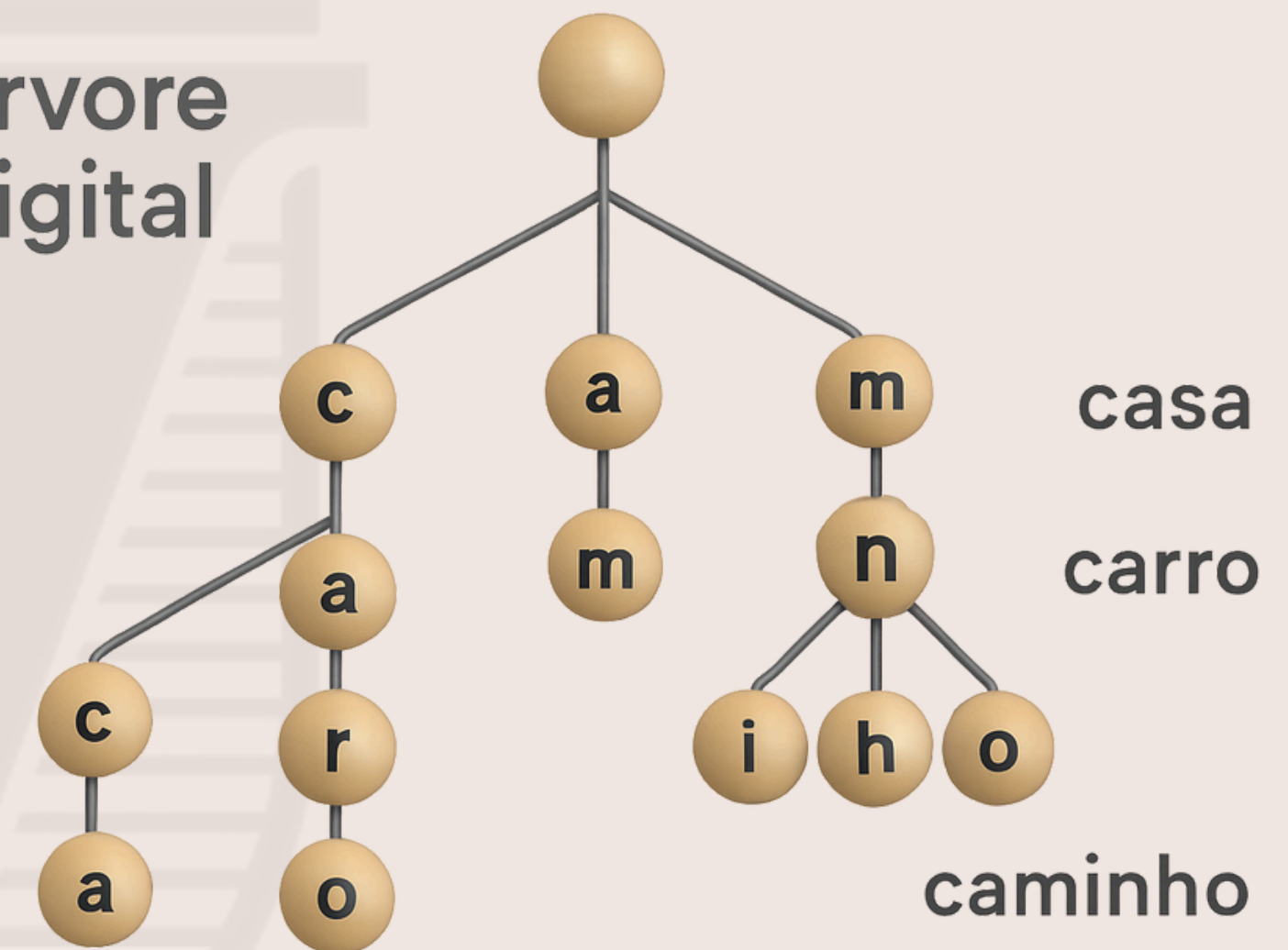
Autocompletar

Verificadores ortográficos

Dicionários

Roteamento IP

Árvore Digital



O que são Árvores de Huffman?

Uma **Árvore de Huffman** é uma estrutura de dados em árvore binária usada para compressão de dados. Foi desenvolvida por David A. Huffman em 1952 enquanto era estudante de doutorado no MIT.

Características principais:

- Utiliza a **frequência de ocorrência** dos símbolos para determinar códigos
- Símbolos mais frequentes recebem códigos mais curtos
- Gera códigos de prefixo (nenhum código é prefixo de outro)

Aplicações

- Compressão de arquivos (ZIP, GZIP)
- Compressão de imagens (JPEG)
- Codificação em telecomunicações

Exemplo

Símbolo	Freq.	Código
A	15	0
B	7	10
C	6	110
D	4	111

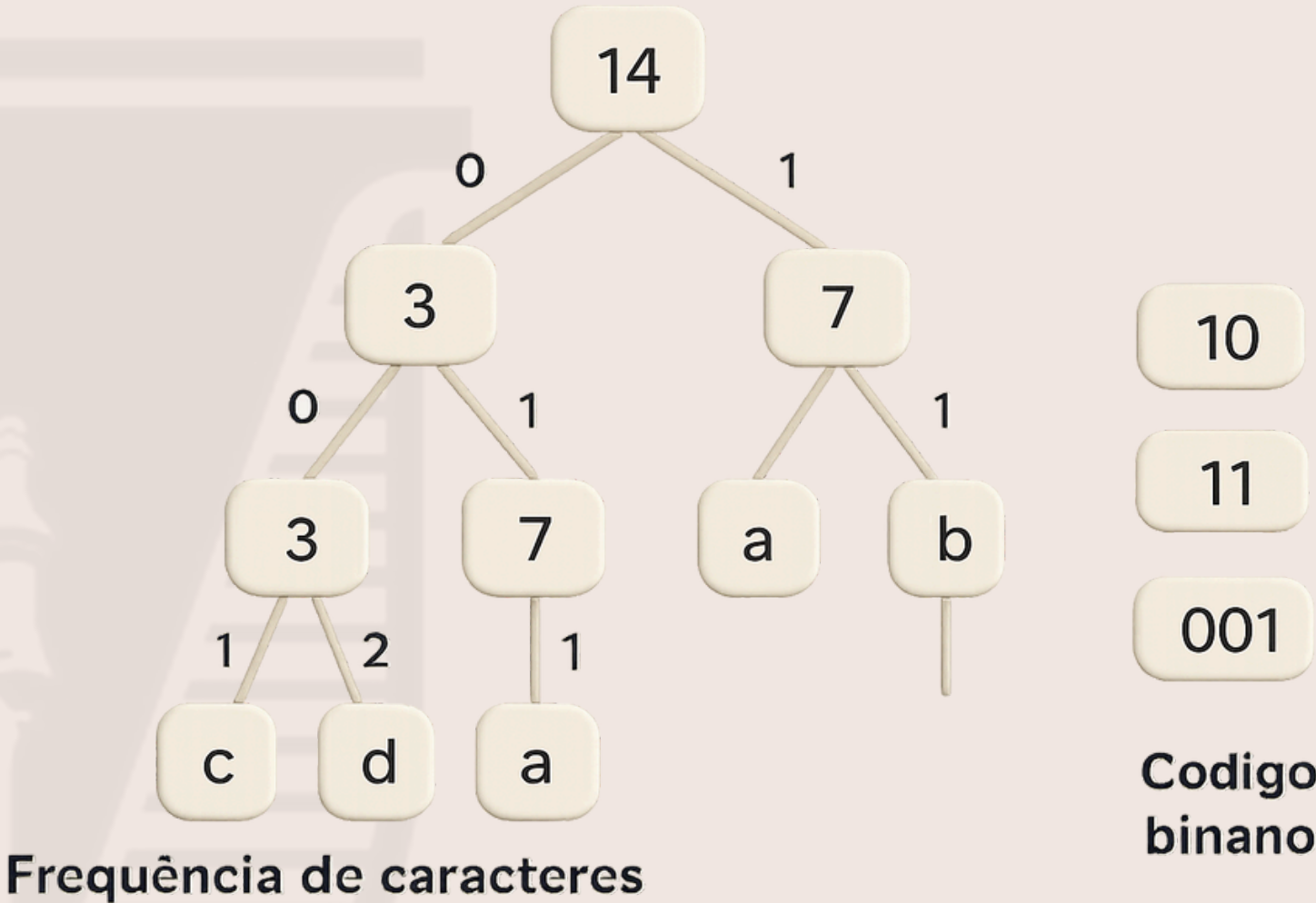
Vantagens:

- Compressão eficiente
- Descompressão rápida
- Sem perda de dados

Desvantagens:

- Requer duas passagens pelos dados
- Árvore armazenada com os dados

Arvore de Huffman



O que são Árvores B?

Uma **Árvore B** é uma estrutura de dados em árvore auto-balanceada que mantém os dados ordenados e permite buscas, inserções e remoções em tempo logarítmico. Foi inventada por Rudolf Bayer e Edward M. McCreight em 1971.

Características principais:

Generalização das árvores binárias de busca

Cada nó pode ter **múltiplos filhos** (não apenas dois)

Cada nó pode armazenar **múltiplas chaves**

Todos os nós folha estão no mesmo nível

Propriedades de uma Árvore B de ordem m:

Cada nó pode ter no máximo m filhos

Cada nó (exceto a raiz) tem pelo menos $\lceil m/2 \rceil$ filhos

Todas as folhas aparecem no mesmo nível

Aplicações

Sistemas de bancos de dados

Índices em bancos de dados

Sistemas de arquivos

Armazenamento em disco

Vantagens:

Minimiza acessos ao disco

Balanceada automaticamente

Ideal para grandes dados

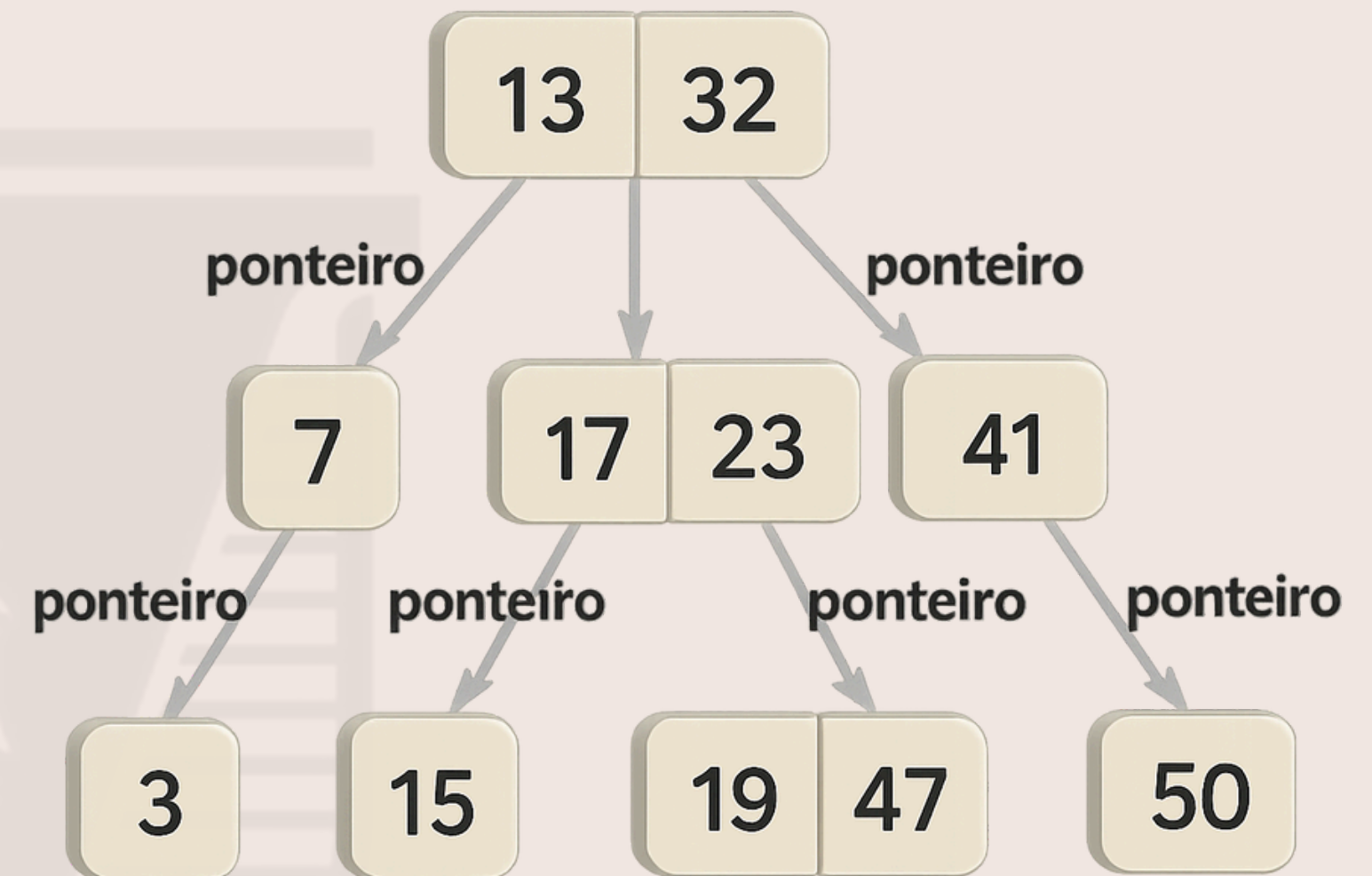
Operações em $O(\log n)$

Desvantagens:

Implementação complexa

Overhead para dados pequenos

Operações mais complexas



O que são Árvores Balanceadas?

Uma **Árvore Balanceada** é uma árvore binária de busca que mantém sua altura mínima possível, garantindo que operações como busca, inserção e remoção sejam realizadas em tempo logarítmico $O(\log n)$.

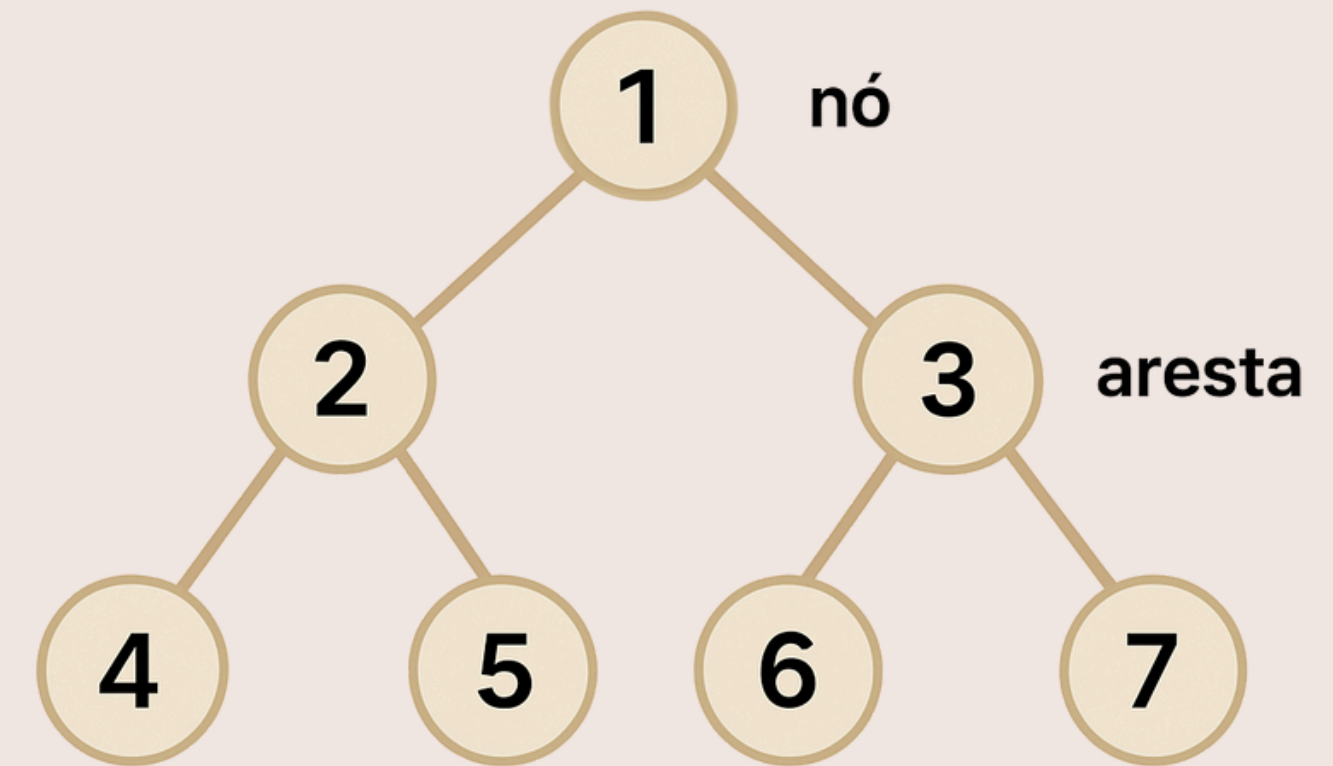
Critérios de balanceamento:

- Balanceamento por altura (AVL)
- Balanceamento por cor (Rubro-Negra)
- Balanceamento por peso (Árvore Peso-Balanceada)
- Balanceamento por tamanho (Árvore B e variantes)

Importância do Balanceamento

Sem balanceamento, uma árvore binária de busca pode degenerar em uma lista ligada, resultando em:

- Operações com complexidade $O(n)$ em vez de $O(\log n)$
- Perda significativa de desempenho para grandes conjuntos de dados
- Uso ineficiente de recursos computacionais



Árvore balanceada

Tipos de Árvores Balanceadas

Onde as Árvores são Utilizadas?

As estruturas de árvores são fundamentais em diversas áreas da computação, oferecendo soluções eficientes para problemas complexos.

Sistemas de Arquivos

Organização hierárquica de diretórios e arquivos.

Compressão de Dados

Algoritmos como Huffman para redução de tamanho.

Compiladores

Análise sintática e árvores de expressão.

Bancos de Dados

Indexação e busca eficiente de registros.

Roteamento em Redes

Determinação de caminhos eficientes em redes.

Inteligência Artificial

Árvores de decisão e algoritmos de busca.



Estruturas de Dados: Árvores

Uma análise dos diferentes tipos de estruturas de árvores

Felipe Casagrande

Bruno Goulart

Diego Cosme

Renata Mendes

Thiago Moura

Bruno Pires

Lucas Borba

Pedro Lucas

Josias Andrade

Eduardo Rodinei